

Mock exam - Object-Oriented Programming

Emily Antosch
HAW Hamburg · Faculty TI

Winter semester 2026/27 · 90 minutes · 105 points

Theory: 60 minutes (Tasks 1–5, 65 points). Programming: 30 minutes (Task 6, 40 points). This practice paper is slightly harder than the actual exam. Use the same time limit to identify topics that need more practice; finish remaining work afterwards. **Instructor copy — do not distribute with the student paper.** Equivalent correct solutions earn full credit. Award partial credit by the criteria shown.

Name:	Student ID:
Date:	

Instructions

- Work individually. Answer Tasks 1–5 in the spaces provided; label any additional sheets with your student ID and task number.
- For Task 6, submit the Java source files in a folder named StudentID_LastName. Use a standard JDK (Java 17 or later), without preview features or external libraries.
- No GUI, interactive menu, file storage, or concurrency is required. Each task is independent.
- You may use two double-sided A4 sheets or four single-sided A4 sheets as your cheat sheet. Only handwritten notes, made with a pen or on a tablet, are permitted. Typed and printed cheat sheets are not allowed.
- Submit both your paper and your programming tasks using the designated drive in Explorer.
- You may start with either theory or programming and switch between them at any time.
- Check that your paper is complete using the page count in the footer.

Tasks

Task 1: Basic Concepts	2
Task 2: Object-Oriented Reasoning	3
Task 3: True or False	4
Task 4: Code Completion	5
Task 5: Code Debugging	7
Task 6: Priority Parcel Queue	9

Task	1	2	3	4	5	6	Total
Available	10	15	16	12	12	40	105
Achieved							

THEORY · 60 MINUTES IN TOTAL

Task 1: Basic Concepts

10 points

Each item earns **1 point for the result or decision** and **1 point for the explanation**. The short snippets are independent and occur inside otherwise valid programs.

a) Pass-by-value (2 points). Box has a public integer field `value`. What does `b.value` contain after the call, and why?

```
1  static void change(Box box) {
2      box.value = 7;
3      box = new Box();
4      box.value = 9;
5  }
6  // Inside main:
7  Box b = new Box(); b.value = 2;
8  change(b);
```

Java

Solution / marking

7 (1). The parameter receives a copy of the reference: the first assignment changes the shared object, but rebinding the parameter does not rebind `b` (1).

b) Class state (2 points). Counter declares `static int created = 0;` and its constructor runs `created++`; . In a fresh run, two instances are constructed and one reference is set to `null`. What is `Counter.created` now, and why?

Solution / marking

2 (1). Both constructors update one shared class field; dropping a reference does not decrement it (1).

c) final references (2 points). Given `final int[] values = {3, 4};`, which is allowed: `values[0] = 8;` or `values = new int[2];`? Explain the distinction.

Solution / marking

Only changing the element is allowed (1). `final` prevents reassignment of the variable, not mutation of the referenced array (1).

d) Strings (2 points). What text is printed, and why?

```
1  String code = "ab";
2  String alias = code;
3  code = code.toUpperCase();
4  System.out.println(alias + "/" + code);
```

Java

Solution / marking

ab/AB (1). Strings are immutable; only `code` is assigned the uppercase result, while `alias` retains the original reference (1).

e) Encapsulation (2 points). A class stores a private `int[]` and its getter returns that array directly. Does `private` alone prevent callers from changing its elements? Explain one way to prevent this exposure.

Solution / marking

No (1). Return a copy such as `return values.clone();` so callers do not receive the internal array (1). An explanation of copying into a new array is also sufficient.

THEORY · 60 MINUTES IN TOTAL

Task 2: Object-Oriented Reasoning

15 points

a) Abstract superclass and super (5 points). Implement a concrete subclass `ExpressTicket` with a constructor `ExpressTicket(int base)` and a `price()` that returns the base price plus 3. Reuse the superclass constructor and getter; do not duplicate the field.

```
1  abstract class Ticket {
2      private int base;
3      public Ticket(int base) { this.base = base; }
4      public int getBase() { return base; }
5      public abstract int price();
6  }
```

Java

Solution / marking

```
1  class ExpressTicket extends Ticket {
2      public ExpressTicket(int base) { super(base); }
3      @Override
4      public int price() { return getBase() + 3; }
5  }
```

Java

Concrete subclass declaration (1); constructor and superclass initialization (2); correct public implementation and result (2).

b) Overload selection and runtime dispatch (5 points). State the two output lines (2 points). Explain which overload is selected at compile time for each call (2 points) and why the first call still runs code in `Loud` (1 point).

```
1  class Announcer {
2      public String say(Object x) { return "A-object"; }
3      public String say(String x) { return "A-string"; }
4  }
5  class Loud extends Announcer {
6      @Override
7      public String say(Object x) { return "L-object"; }
8  }
9  // Inside main:
10 Announcer a = new Loud();
11 Object message = "hello";
12 System.out.println(a.say(message));
13 System.out.println(a.say("hello"));
```

Java

Solution / marking

Output: L-object, then A-string (1 each). The argument's declared type selects `say(Object)` for message (1). The string literal selects the more specific `say(String)` (1). The selected `say(Object)` instance method is overridden in the runtime type `Loud` (1).

c) Interface reference (5 points). An interface `Resettable` declares only `void reset()`. The concrete class `Meter` implements `Resettable` has public methods `void reset()` and `int level()`. Given `Resettable r = new Meter();`, does `r.reset()` compile? Does `r.level()` compile? Explain each decision (2 points each). Give a reference declaration that lets you call both methods on a new `Meter` **without a cast** (1 point).

Solution / marking

`r.reset()` compiles because the declared interface includes it (2). `r.level()` does not: `Resettable` does not declare `level`, even though the object is a `Meter` (2). `Meter m = new Meter();` makes both methods available (1).

THEORY · 60 MINUTES IN TOTAL

Task 3: True or False

16 points

Mark exactly one column for each statement. Each correct answer earns **2 points**. An incorrect, missing, or double-marked answer earns 0 points; there are no deductions.

No.	Statement	True	False
1	A concrete subclass may satisfy an interface's abstract method by inheriting a matching public implementation from its superclass.	X	
2	If a variable has an interface type, it can call every public method of the object's concrete class without a cast.		X
3	A class can be abstract even if it declares no abstract methods.	X	
4	Declaring a field <code>private</code> makes the referenced object immutable.		X
5	Outside the declaring package, every class may access a protected instance field through any superclass reference.		X
6	Two overloads may differ in parameter types while having the same return type.	X	
7	An overriding instance method may reduce visibility from <code>public</code> to <code>protected</code> .		X
8	If both operands of <code>/</code> are integers, assigning the result to a <code>double</code> does not restore a discarded fractional part.	X	

Solution / marking

1. True: an inherited public implementation can satisfy the contract.
2. False: the declared reference type limits compile-time member access.
3. True: the abstract modifier alone prevents direct instantiation.
4. False: `private` controls field access, not the mutability of its value.
5. False: cross-package protected access is restricted to subclass contexts, with receiver restrictions for instance members.
6. True: different parameter types distinguish the overloads.
7. False: an override cannot reduce access.
8. True: integer division occurs before widening to `double`.

THEORY · 60 MINUTES IN TOTAL

Task 4: Code Completion

12 points

Complete gaps **A–F** directly in the listing (**2 points per gap**). `ScoreRange` represents an inclusive integer interval. Both constructor arguments are guaranteed to lie between -1000 and 1000 , but may arrive in the wrong order. Reject `low > high` with `IllegalArgumentException`; equal endpoints are valid. `contains` includes both endpoints. `clamp` returns the nearest endpoint for an out-of-range value, or the value itself when it is inside. The exact string format is `ScoreRange[2..5]` for endpoints 2 and 5.

```
1      public class ScoreRange {
2          // A: declare two private final int fields
3          -----
4          -----
5      public ScoreRange(int low, int high) {
6          // B: reject reversed endpoints
7          -----
8          -----
9          // C: initialize both fields
10         -----
11         -----
12     }
13     public boolean contains(int value) {
14         // D
15         -----
16     }
17     public int clamp(int value) {
18         // E
19         -----
20         -----
21         -----
22     }
23     @Override
24     public String toString() {
25         // F
26         -----
27     }
28 }
```

Java

Solution / marking

```
1    public class ScoreRange {
2        private final int low;
3        private final int high;
4        public ScoreRange(int low, int high) {
5            if (low > high) throw new IllegalArgumentException();
6            this.low = low;
7            this.high = high;
8        }
9        public boolean contains(int value) {
10           return value ≥ low && value ≤ high;
11        }
12        public int clamp(int value) {
13            if (value < low) return low;
14            if (value > high) return high;
15            return value;
16        }
17        @Override
18        public String toString() {
19            return "ScoreRange[" + low + ".." + high + "]";
20        }
21    }
```

Java

A: one point per private final field. B: correct condition (1), exception (1). C: one point per field assignment. D: inclusive comparisons (1), conjunction (1). E: correct behavior below/above the range (1), unchanged in-range value (1). F: both field values (1), exact punctuation/order (1).

THEORY · 60 MINUTES IN TOTAL

Task 5: Code Debugging

12 points

Find **six independent defects**. For each, identify the location, explain the problem, and give a correction (**1 point for the explanation, 1 for the correction**). All fields must be private. The constructor must retain the code and initial number of seats. `matches` must compare code text, and `reserve` must return `false` for zero remaining seats without changing state; otherwise it must remove exactly one seat and return `true`. Assume non-null codes and a nonnegative initial seat count. No additional validation or getters are required.

```
1      public class Booking {
2          public String code;
3          private int remaining;
4          public Booking(String code, int remaining) {
5              code = code;
6              this.remaining = remaining;
7          }
8          public static int getRemaining() {
9              return remaining;
10         }
11         public boolean matches(String other) {
12             return code == other;
13         }
14         public boolean reserve() {
15             if (remaining < 0) {
16                 return false;
17             }
18             remaining++;
19             return true;
20         }
21     }
```

Java

Solution / marking

1. code is public, contrary to the specification; declare it private.
2. code = code assigns the parameter to itself; use `this.code = code;`.
3. A static `getRemaining` cannot access this instance's field directly; remove static.
4. `=` tests references rather than text; use `code.equals(other)`.
5. The guard misses zero seats; use `remaining ≤ 0` (or `= 0` under the stated invariant).
6. Incrementing adds a seat; use `remaining--;`.

Corrected reference implementation:

```
1  public class Booking {
2      private String code;
3      private int remaining;
4      public Booking(String code, int remaining) {
5          this.code = code;
6          this.remaining = remaining;
7      }
8      public int getRemaining() { return remaining; }
9      public boolean matches(String other) { return code.equals(other); }
10     public boolean reserve() {
11         if (remaining ≤ 0) return false;
12         remaining--;
13         return true;
14     }
15 }
```

Java

PROGRAMMING · 30 MINUTES

Task 6: Priority Parcel Queue

40 points

Build a console application that stores parcels awaiting dispatch. Use three files: `Parcel.java`, `ParcelQueue.java`, and `Demo.java`. Use an `ArrayList`, not a priority-queue library. No keyboard input, sorting, persistence, or delivery-history collection is required.

Data and behavior

- A `Parcel` stores an ID (`String`) and priority (`int`, **1 through 3 inclusive**). Higher numbers mean higher priority. It has no methods that change those values after construction.
- The `Parcel` constructor receives valid values. `ParcelQueue.addParcel` performs validation.
- A valid ID is non-null and not empty after `trim()`. Store the trimmed ID. IDs in the current queue must be unique **ignoring case**. After dispatch, an ID may be used again.
- `addParcel(String id, int priority)` returns `false` for invalid input or a duplicate ID, leaving the queue unchanged. Otherwise it appends a parcel and returns `true`.
- `findParcel(String id)` trims the search ID and matches ignoring case. It returns the stored parcel or `null` if no match exists or the ID is null/blank. It must not remove anything.
- `dispatchNext()` removes and returns the highest-priority parcel, or returns `null` when empty. For equal priorities, dispatch the one added **earliest**.
- `displayAll()` prints the parcels in insertion order, or `No parcels` when empty.

Implementation and points

Part	Deliverable	Points
a	Create the named project folder and three Java files with matching public classes.	3
b	<code>Parcel</code> : two private fields, constructor, both getters, and a readable <code>toString()</code> containing both values.	5
c	<code>ParcelQueue</code> : private <code>ArrayList<Parcel></code> initialized as an empty list.	4
d	Implement boolean <code>addParcel(String id, int priority)</code> including validation and duplicate rejection.	6
e	Implement <code>Parcel findParcel(String id)</code> with normalization, case-insensitive matching, and no-match behavior.	5
f	Implement <code>Parcel dispatchNext()</code> including ties and empty-queue behavior.	6
g	Implement void <code>displayAll()</code> including the empty message.	3
h	<code>Demo.main</code> : demonstrate the acceptance checks on the next page using labelled output or assertions.	5
i	Use clear names, consistent indentation, and encapsulation; keep queue operations in <code>ParcelQueue</code> , not <code>Demo</code> .	3

Task 6: Acceptance checks and API reminder

Use a fresh queue for this sequence:

1. Display the empty queue. Searching and dispatching must return `null`.
2. Add (" P1 ", 2), ("P2", 3), and ("P3", 3) successfully, then display them in insertion order. The first stored ID must be P1 without surrounding spaces.
3. Reject ("p1", 1), a null ID, a spaces-only ID, and priority 4. Finding " p2 " must return the stored P2 parcel without removing it; an unknown ID must return `null`.
4. Dispatch all parcels: the IDs must be P2, then P3, then P1. The next dispatch must return `null`. Invalid additions must not have changed the dispatch order or parcel count.
5. Add ("p1", 1) again successfully and check that it can now be found and dispatched.

`ArrayList` needs `import java.util.ArrayList;`. These operations may help:

```
1  ArrayList<Parcel> parcels = new ArrayList<>();
2  parcels.add(new Parcel("X", 1));
3  Parcel first = parcels.get(0);
4  Parcel removed = parcels.remove(0); // removes by index and returns it
5  int size = parcels.size();
6  boolean empty = parcels.isEmpty();
7  // For non-null strings:
8  boolean same = "P1".equalsIgnoreCase("p1");
9  String trimmed = " P1 ".trim();
```

Java

When choosing a parcel, remember the index of the best candidate. Do not modify the list while searching for that candidate. Submit all three `.java` files. A compiling partial solution can earn credit for the parts it implements correctly.

Solution / marking

Parcel.java

```
1  public class Parcel {
2      private final String id;
3      private final int priority;
4      public Parcel(String id, int priority) {
5          this.id = id;
6          this.priority = priority;
7      }
8      public String getId() { return id; }
9      public int getPriority() { return priority; }
10     @Override
11     public String toString() { return id + ": priority " + priority; }
12 }
```

Java

ParcelQueue.java

```
1  import java.util.ArrayList;
2  public class ParcelQueue {
3      private final ArrayList<Parcel> parcels = new ArrayList<>();
4      public boolean addParcel(String id, int priority) {
5          if (id == null || id.trim().isEmpty()
6              || priority < 1 || priority > 3
7              || findParcel(id) != null) {
8              return false;
9          }
10         parcels.add(new Parcel(id.trim(), priority));
11         return true;
12     }
13     public Parcel findParcel(String id) {
14         if (id == null || id.trim().isEmpty()) return null;
15         for (Parcel p : parcels) {
16             if (p.getId().equalsIgnoreCase(id.trim())) return p;
17         }
18         return null;
19     }
20     public Parcel dispatchNext() {
21         if (parcels.isEmpty()) return null;
22         int best = 0;
23         for (int i = 1; i < parcels.size(); i++) {
24             if (parcels.get(i).getPriority()
25                 > parcels.get(best).getPriority()) {
26                 best = i;
27             }
28         }
29         return parcels.remove(best);
30     }
31     public void displayAll() {
32         if (parcels.isEmpty()) System.out.println("No parcels");
33         for (Parcel p : parcels) System.out.println(p);
34     }
35 }
```

Java

Demo.java (run with `java -ea Demo` to enable assertions).

```

1      public class Demo {
2          public static void main(String[] args) {
3              ParcelQueue queue = new ParcelQueue();
4              queue.displayAll();
5              assert queue.findParcel("none") == null;
6              assert queue.dispatchNext() == null;
7              assert queue.addParcel(" P1 ", 2);
8              assert queue.addParcel("P2", 3);
9              assert queue.addParcel("P3", 3);
10             assert queue.findParcel("P1").getId().equals("P1");
11             queue.displayAll();
12             assert !queue.addParcel("p1", 1);
13             assert !queue.addParcel(null, 2);
14             assert !queue.addParcel("  ", 2);
15             assert !queue.addParcel("P4", 4);
16             assert queue.findParcel(" p2 ").getId().equals("P2");
17             assert queue.findParcel("missing") == null;
18             assert queue.dispatchNext().getId().equals("P2");
19             assert queue.dispatchNext().getId().equals("P3");
20             assert queue.dispatchNext().getId().equals("P1");
21             assert queue.dispatchNext() == null;
22             assert queue.addParcel("p1", 1);
23             assert queue.findParcel("P1") != null;
24             assert queue.dispatchNext().getId().equals("p1");
25         }
26     }

```

Java

Marking (40 points): a: folder (1), classes/files (2). b: private fields (1), constructor (1), getters (2), string (1). c: private typed list (2), empty initialization (2). d: null/blank validation (1), bounds (1), trim (1), duplicate rejection (1), add/true (1), unchanged/false (1). e: null/blank guard (1), trimmed case-insensitive comparison (2), stored object/no-match returns (2). f: empty guard (1), maximum selection (2), stable ties (1), removal and returned parcel (2). g: all parcels in order (2), empty message (1). h: one point for each of the five acceptance checks. i: naming (1), formatting (1), responsibility separation/encapsulation (1). Accept equivalent loop solutions; no sorting is required. Do not deduct twice for one underlying error.